Student Cyber Security Management System

Final Project Report

Python and C Programming Internship
White Band Associates
Final Task 06

1. Objective

The purpose of this final project is to bring together the concepts covered throughout the internship — variables, input/output, conditional statements, loops, functions, arrays/lists, and file handling — into a single working application rather than a set of separate small programs. The application, a Student Cyber Security Management System, manages student records and evaluates a basic security posture for each student, with all data persisted to a text file so it survives between runs.

2. Features

The system is fully menu-driven and offers the following options:

- Add Student – stores Student ID, Name, Branch, and Email
- View Students – displays every saved record along with security score and status
- Search Student – search by Name or by Student ID
- Delete Student – removes a record after a yes/no confirmation
- Security Assessment – asks about MFA, password length, system updates, and antivirus, then calculates a Security Score out of 100 with a status of Excellent, Good, Moderate, or Poor
- Generate Report – shows total students, individual scores, the average score, and a list of students with a Poor rating
- Password Strength Checker – rates a password as Weak, Moderate, or Strong
- Username Generator – generates a username from a name and birth year
- Login Validation – a basic username/password check simulating a login system
- Exit – closes the program safely

3. Technologies Used

Component	Details
Language	Python 3
Storage	Plain text file (students.txt), pipe-delimited records
Built-in Modules	re (used for password strength validation)
Dependencies	None – standard library only

4. Program Flow

The program is built around one shared list of student records, loaded once when the program starts and written back to students.txt every time it changes. The overall flow is:

```
Program starts → load students.txt into memory → display main menu in a loop → user selects an option → the matching function runs and updates the in-memory list → if the data changed, it is saved back to students.txt immediately → the loop repeats until the user selects Exit.
```

The Security Assessment module deserves a special mention: it looks up a student by ID, asks the four security questions, computes a weighted score (25 points each for MFA, password length, system updates, and antivirus — with password length scored on a sliding scale based on character count), and writes the resulting score and status directly back into that student's record. The Generate Report module then simply reads these saved scores rather than recalculating anything.

Security Score → Status Mapping

Score Range	Status
90 – 100	Excellent
70 – 89	Good
50 – 69	Moderate
Below 50	Poor

5. Challenges Faced

- Keeping the file format simple but reliable – a pipe-separated (|) text format was chosen over something like JSON since it's easier to read and write line by line, but it meant being careful that no stored field would ever contain a | character.
- Handling students who haven't been assessed yet – View Students, Generate Report, and the average score calculation all had to safely handle a security score of "None" without crashing.
- Validating user input – password length needed to be a whole number, so a retry loop was added instead of letting bad input crash the program.
- Designing the scoring logic – deciding how many points each security factor should contribute so the 0–100 range mapped sensibly onto Excellent/Good/Moderate/Poor.
- Preventing duplicate Student IDs – since IDs are typed in manually rather than auto-generated, a check was added before adding a new student.

6. Learning Outcomes

- Designing a program around one shared data structure (a list of dictionaries) that every function reads from and writes to, instead of writing isolated scripts.
- Practical file handling – not just a single read/write, but keeping a text file in sync with in-memory data across many operations in a single running session.
- Writing small, focused, reusable functions (for example, separating score calculation from the function that asks the security questions), which made debugging much easier.
- A better understanding of basic security concepts – MFA, password strength, patching, and antivirus status – and how a simple weighted scoring system can be used to flag risk.
- The importance of input validation and error handling once a program becomes interactive instead of running on fixed sample data every time.

7. Conclusion

This final project pulled together the full range of concepts covered during the internship into one cohesive, menu-driven application. Rather than aiming for a complex feature set, the focus was on clean, well-structured code, sensible error handling, and a clear understanding of how each module works — which was the actual goal of this task.